

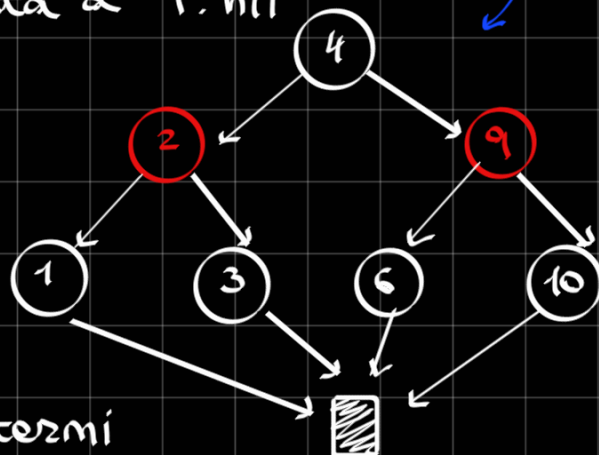
RIPASSO ALBERI ROSSO-NERI (RBT)

Un **RBT** è un BST i cui nodi sono dotati di un attributo aggiuntivo, detto **COLORE** e $\{\text{ROSSO}, \text{NERO}\}$, e soddisfacente le seguenti 5 proprietà:

- ① Ogni nodo è **ROSSO** o **NERO**
- ② la radice è **NERA**
- ③ le foglie sono **NERE**
- ④ I figli di un nodo **ROSSO** sono entrambi **NERI**
- ⑤ Per ogni nodo dell'albero, tutti i cammini dai suoi discendenti alle foglie contenute nei suoi sottoalberi hanno lo stesso numero di nodi **NERI**

CONVENZIONI:

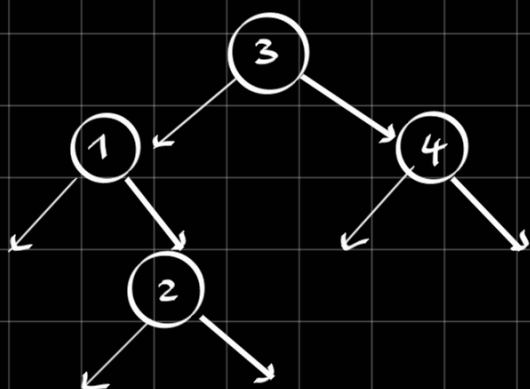
- le foglie non possiedono dati: sono tutte **NIL**. Per semplificare, le rappresentiamo tutte come un unico nodo con riferimento **T.nil**
- Il padre del nodo radice punta a **T.nil**



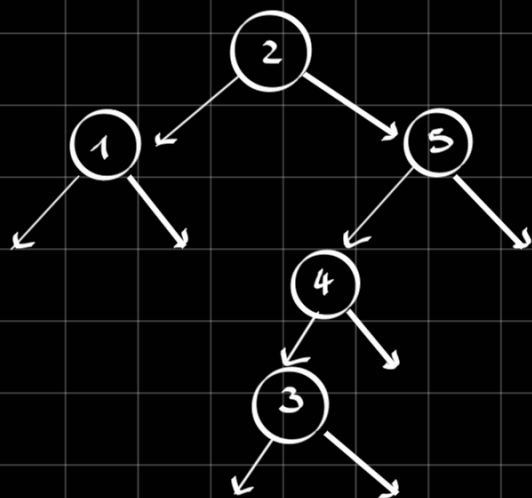
PROPRIETÀ RBT:

- Un albero RB con n nodi interni ha altezza massima $2\log(n+1)$
- Quindi le operazioni **RICERCA**, **MAX**, **MIN** e **SUCCESSORE** sono $O(2\log(n+1))$
- Anche le operazioni per riparare le proprietà degli alberi, **INSERISCI** e **CANCELLA**, sono $O(2\log(n+1))$

ESEMPI:



Non è un RBT: non vale la ⑤. Per sistemarlo, possiamo colorare il modo 2 di rosso



Anche qui, la proprietà ⑤ è violata:

- non posso colorare 5 o 1 di rosso

- se coloro 4, non posso colorare 3

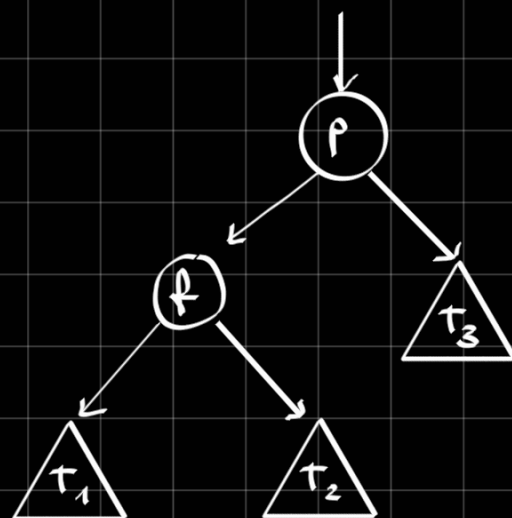
⇒ L'albero è troppo sbilanciato e non ammette colorazione valida

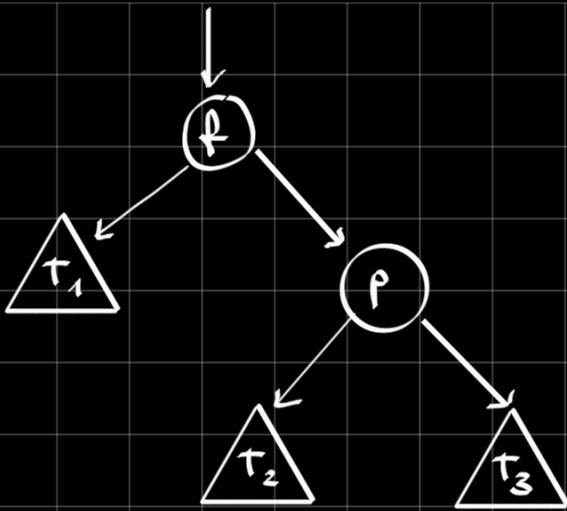
⇒ Per rendere l'albero un RBT devo modificare la posizione dei nodi! Per farlo si usa la tecnica di **ROTAZIONE**

Supponiamo di avere questo sottoalbero (sbilanciato). Per bilanciarlo, facciamo una **ROTAZIONE DESTRA**:

- Scambio p e f , mettendo p a destra

- Riposiziono gli altri sottoalberi mantenendo le proprietà dei BST...

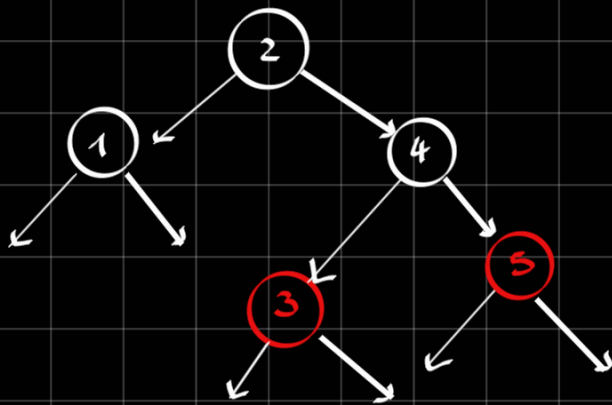




Il risultato dell'operazione è la seguente. Notiamo che la profondità di T_1 è diminuita di 1 nodo meno, la profondità di T_3 è aumentata e quella di T_2 è invariata.

L'operazione inversa è una ROTAZIONE SINISTRA

Per bilanciare l'albero di prima, dobbiamo fare una rotazione destra tra 5 e 4.



A questo punto, basta colorare di rosso 3 e 5 per far valere tutte le proprietà.

vediamo degli esercizi di sintesi di algoritmi sui RBT

1

Si descriva un algoritmo che prende in input un BST T i cui nodi hanno un attributo aggiuntivo *color* (che vale *RED* oppure *BLACK*), e restituisce *true* se T è un RBT, *false* altrimenti.

SOLUZIONE:

Basta scrivere un algoritmo che verifica tutte le 5 proprietà viste prima. Per quanto scritto nel problema, la ① è già verificata.

Inoltre, poiché tutte le foglie sono racchiuse in $T.nil$, per verificare la ③ basta verificare che $T.nil.color = BLACK$.

Per le ultime 2 proprietà scriviamo una procedura ricorsiva.

ISRBT (T):

if $T = NIL$: return true

if $T.root.color = RED$:
return false

if $T.nil = RED$:
return false

(altezza, valido) $\leftarrow$ AUX ($T.root, T$)
return valido

se l'albero è vuoto $\Rightarrow$ TRUE

Proprietà ② e ③

AUX (x, T):

if $x = T.nil$:
return (1, true)

if $x.color = RED$:

if $x.left.color = RED$ or $x.right.color = RED$:
return (-1, false)

Se sto in una foglia ritorno due valori: 1 (l'altezza di questo albero) e true (è un RBT)

controllo condizione ④

Se ho superato i controlli precedenti, verifico la stessa condizione sui figli...

$$\begin{aligned}(\text{altezzaleft}, \text{isRBleft}) &\leftarrow \text{AUX}(x.\text{left}, T) \\ (\text{altezzaright}, \text{isRBright}) &\leftarrow \text{AUX}(x.\text{right}, T)\end{aligned}$$

if $\neg \text{isRBleft}$ or $\neg \text{isRBright}$ or
 $\text{altezzaleft} \neq \text{altezzaright}$:
 return $(-1, \text{false})$

if $x.\text{color} = \text{RED}$:
 return $(\text{altezzaleft}, \text{true})$
else:
 return $(\text{altezzaleft} + 1, \text{true})$

Notiamo che questo algoritmo, pur avendo tanti if statement e altre operazioni, risulta essere una visita dell'albero (che può terminare prima se uno degli if statement mi restituisce false). Quindi ha complessità $O(n)$

2) Progettare una struttura dati che gestisce un insieme dinamico (inserimenti e rimozioni) in cui si possa eseguire $\text{selezione}(k)$, che restituisce il k -esimo elemento più piccolo.
Tutte le operazioni andrebbero fatte in $O(\log(n))$, dove n è la dimensione della struttura usata.

SOLUZIONE:

Le strutture dati già incontrate possono aiutare parzialmente, permettendo di fare molto velocemente alcune (ma non tutte!) operazioni

C'è una struttura più utile?

IDEA: Conservo i dati in un **ALBERO ROSSO-NERO**, dove inserimenti e rimozioni costano $O(\log(n))$, la selezione la faccio con una visita in ordine dei primi k elementi: $O(n) \rightarrow$ non va bene!

Un modo di risolvere questo problema è usare un RBT "arricchito", dove ogni nodo ha un campo "size" che contiene il numero di nodi del sottoalbero ivi radicato.

Tuttavia, devo garantire che questa informazione possa essere conservata senza aumentare le complessità di insert/remove

Questo fatto è garantito dal **TEO. DI CORMEN E LEISERSON**

TEO. DI CORMEN - LEISERSON

Se ogni nodo di un RBT ha un nuovo campo "val" tale che per ogni x si ha che $x.val$ dipende solo dai dati presenti in $x.left$ e $x.right$ e può essere calcolato in $O(1)$, allora insert/remove restano in $O(\log(n))$

OSSERVA: $x.size = x.left.size + x.right.size + 1$
rispetta il teorema

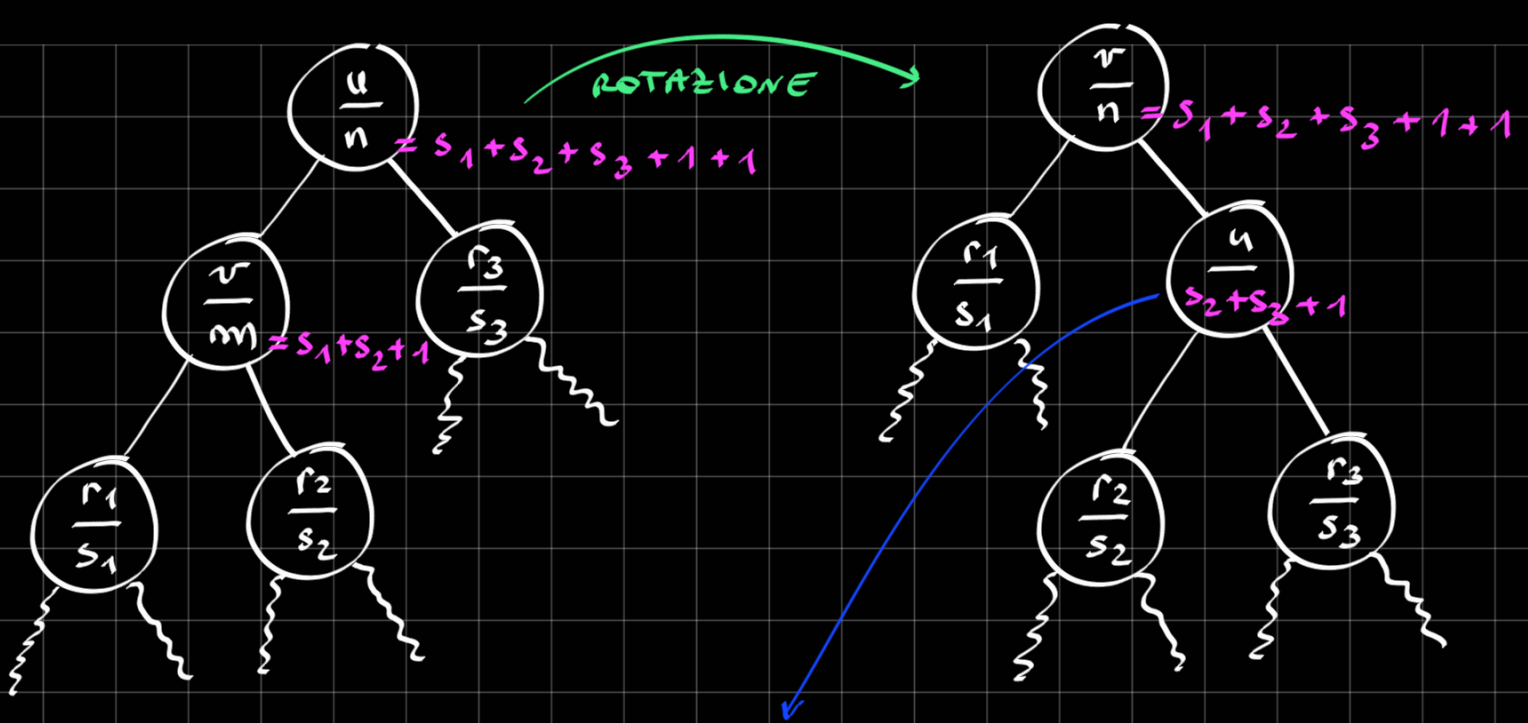
Ma quindi potrei salvare direttamente in ogni nodo un valore "rank" in modo che $x.rank$ sia la posizione di x durante la visita in ordine?

RISPOSTA: NO: il campo "rank" non soddisfa il teorema, in quanto $x.rank$ non dipende solo da $x.left$ e $x.right$

DOMANDA: Come gestire size la insert?

- Durante la fase di ricerca della posizione in cui inserire il nodo nuovo, basta incrementare di 1 la size di ogni nodo visitato (in discesa)
 $O(\log(n))$

- Segue un numero costante di rotazioni che serve per ripristinare le proprietà dei RBT: bisogna aggiornare solo la size dei nodi che vengono ruotati $O(1)$



Questo valore nuovo lo calcolo
in $O(1)$

PSEUDOCODICE:

selezione (root, k) :

x = root

finché $x \neq \text{NIL}$:

se $x.\text{left} \neq \text{NIL}$

$r = x.\text{left}.\text{size}$

altrimenti $r = 0$

rango_corrente = $r + 1$

se $k = \text{rango_corrente}$

return x

altrimenti se $k < \text{rango_corrente}$

$x = x.\text{left}$

altrimenti

$k = k - \text{rango_corrente}$

$x = x.\text{right}$

return NULL

3) Ho un flusso costante di dati (interi), e voglio tenere traccia solo degli ultimi m arrivati, potendo trovare velocemente il minimo e massimo degli ultimi m e quanti ce ne sono di compresi tra a e b (con $a \leq b$)

Voglio che tutte queste operazioni si possano fare in $O(\log(n))$

SOLUZIONE:

Se non dovessimo fare le "query", la struttura più ovvia da usare sarebbe una coda FIFO. Senza cronologia userei un RBT con size.

Ma in questo caso abbiamo entrambe le cose. Come facciamo?

IDEA: Combino le due strutture scritte prima

- Una CODA che conserva gli n elementi più recenti
- L'RBT con size usa i dati come chiave (anche l'RBT ha dimensione $= m+1$)

Gli elementi della coda vanno collegati al RBT; in coda abbiamo dei puntatori a un nodo dell'RBT (l'elemento in testa alla coda indica l'elemento inserito meno recentemente...)

l'insert funzionerà così:

- inserisco nell'RBT un nuovo nodo u con chiave x e campo "size" calcolato opportunamente
- inserisco in coda un puntatore a u
- rimuovo dall'RBT il nodo puntato dall'elemento in testa alla coda
- rimuovo la testa dalla coda

Tutte queste operazioni insieme costano $O(\log m)$

Invece...

Ricerca min/max:

- min: scendo dalla radice sempre a sx
- max: scendo dalla radice sempre a dx

In entrambi i casi, l'operazione costa $O(\log m)$

Per calcolare il # di el. in $[a, b]$...

conta_minori($root, c$)

$x = root$

conta = 0

finché $x \neq NIL$

se $x.left \neq NIL$

sin = $x.left.size$

altrimenti sin = 0

se $c \leq x.key$ ←

$x = x.left$

Quelli a dx di x
sono tutti $\geq c$

altrimenti $\leftarrow$ $c > x.key \rightarrow$ conto tutto
il sottoalbero radicato in x
 $conta = conta + \text{sin} + 1$
 $x = x.right$

OSSERVA: Questa procedura funziona anche se
ci sono duplicati: per contare quelli $\leq c$ basta
fare $\text{conta}(root, c+1)$

Alla fine di tutto, la complessità sarà un
 $O(\log n)$, perché nel caso pessimo visiteremo
ogni livello del RBT
